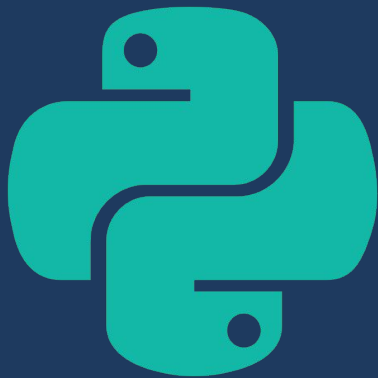


DAY 2 - AI Workshop



Python Basics

for Artificial Intelligence

Variables • Data Types • Control Flow • Functions • NumPy

Session Overview

Why Python for AI?

Ecosystem overview, setup, REPL basics

Control Flow

if/elif/else statements and loops (for, while)

Variables & Data Types

int, float, str, list, dict, tuple

Functions & Modules

Defining functions, parameters, return values

Operators & Expressions

Arithmetic, comparison, logical, membership

Intro to NumPy

Arrays, operations, why it matters for AI

Why Python for AI & Machine Learning?

The go-to language for data science and AI development

Simple Syntax

Reads like English. Beginners master it fast; experts stay productive.

Huge Ecosystem

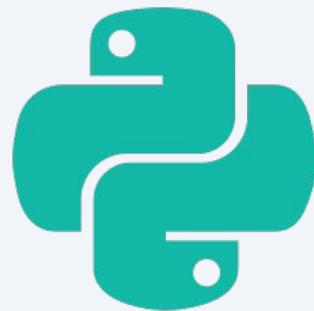
NumPy, Pandas, Scikit-learn, TensorFlow, PyTorch — all Python.

Community

Millions of developers; endless tutorials, forums, and open-source code.

Cross-Platform

Runs on Windows, macOS, Linux — and in the cloud (Colab, Jupyter).



#1

Most Popular
Programming
Language (TIOBE)

Variables & Data Types

How Python stores and manages information

```
python variables.py
```

```
# Variables — no type declaration needed!
name = "Alice"          # str
age = 21                 # int
gpa = 3.85               # float
is_student = True       # bool

# Collections
scores = [95, 87, 92]    # list
student = {"name": "Alice", # dict
           "age": 21}
coords = (10.5, 20.3)     # tuple

# Type checking
print(type(name))        # <class 'str'>
print(type(scores))      # <class 'list'>
```

int / float

```
age = 21
gpa = 3.85
```

list

```
scores = [95, 87]
```

str

```
name = "Alice"
```

dict

```
{"key": "val"}
```



Python is dynamically typed — variables can change type at runtime. Use descriptive names (snake_case) for readability.

Control Flow — if / for / while

Making decisions and repeating tasks in Python

```
python conditionals.py
```

```
# if / elif / else
score = 78

if score >= 90:
    grade = "A"
elif score >= 80:
    grade = "B"
elif score >= 70:
    grade = "C"
else:
    grade = "F"

print(f"Grade: {grade}")    # Grade: C
```

```
python loops_for.py
```

```
# for loop
fruits = ["apple", "banana", "mango"]

for fruit in fruits:
    print(fruit.upper())

# range
for i in range(1, 6):
    print(i)    # 1 2 3 4 5
```

```
python loops_while.py
```

```
# while loop
count = 0
while count < 5:
    print(count)
```



Key Points

- Python uses indentation (4 spaces) instead of curly braces — indentation IS the code block
- Use for loops when you know iterations in advance; while loops when you don't
- range(start, stop, step) is your best friend for numeric loops

Functions & Modules

Reusable code blocks and importing libraries

```
python functions.py
```

```
# Defining a function
def calculate_bmi(weight, height):
    """Calculate Body Mass Index."""
    bmi = weight / (height ** 2)
    return round(bmi, 2)

# Calling a function
my_bmi = calculate_bmi(70, 1.75)
print(f"BMI: {my_bmi}")    # BMI: 22.86

# Default parameters
def greet(name, lang="English"):
    if lang == "Nepali":
        return f"Namaste, {name}!"
    return f"Hello, {name}!"

print(greet("Sita"))        # Hello, Sita!
print(greet("Ram", "Nepali")) # Namaste, Ram!
```

Importing Modules

```
import math
```

sqrt, pi, log, sin, cos...

```
import random
```

random, randint, shuffle...

```
import os / sys
```

File paths, env variables

```
import datetime
```

Date/time manipulation

Introduction to NumPy

Fast numerical computing — the backbone of AI libraries

```
python numpy_intro.py
```

```
import numpy as np

# Create arrays
a = np.array([1, 2, 3, 4, 5])
b = np.zeros((3, 3))      # 3x3 matrix of zeros
c = np.ones((2, 4))       # 2x4 matrix of ones
d = np.arange(0, 10, 2)   # [0, 2, 4, 6, 8]

# Element-wise operations (fast!)
x = np.array([1, 2, 3])
y = np.array([4, 5, 6])
print(x + y)   # [5, 7, 9]
print(x * y)   # [4, 10, 18]
print(x ** 2)  # [1, 4, 9]

# Useful stats
print(np.mean(a))   # 3.0
print(np.std(a))     # 1.414
print(np.max(a))     # 5
```

Why NumPy for AI?



100x Faster

C-backed arrays vs Python lists



Matrix Math

Dot products, transpose, reshape



Foundation

Pandas, Scikit-learn, TensorFlow all use NumPy arrays internally



Broadcasting

Operations on arrays of different shapes

Data Handling & Visualization

Pandas • CSV Datasets • Data Cleaning • Matplotlib • Seaborn • EDA



Session Overview

From raw data to meaningful visual insights

Pandas Introduction

Series, DataFrames, reading CSV files

Data Visualization

Matplotlib & Seaborn — line, bar, scatter, heatmap

Working with CSV Datasets

head(), info(), describe(), indexing & selection

Exploratory Data Analysis

Correlation, distributions, outlier detection

Data Cleaning

Missing values, duplicates, type conversion

Mini EDA Demo

Full analysis on a real dataset (iris/titanic)

Introduction to Pandas

Python's most powerful data manipulation library

```
python pandas_intro.py
import pandas as pd

# Series – 1D labeled array
scores = pd.Series([85, 92, 78, 95],
                    index=["Alice", "Bob", "Charlie", "Diana"])
print(scores["Alice"])    # 85

# DataFrame – 2D table (like Excel)
data = {
    "Name":  ["Alice", "Bob", "Charlie"],
    "Age":   [21, 23, 22],
    "Score": [85, 92, 78]
}
df = pd.DataFrame(data)
print(df)
#      Name  Age  Score
# 0   Alice   21     85
# 1    Bob    23     92
# 2  Charlie   22     78

# Read from CSV
df = pd.read_csv("students.csv")
```

Two Core Structures

Series

A one-dimensional labeled array.
Like a single column of a spreadsheet.
Supports any data type.

DataFrame

A two-dimensional table with rows and columns.
Like an Excel sheet in Python.
Built on top of NumPy arrays.

Working with CSV Datasets

Essential operations for exploring and querying data

```
python csv_explore.py
```

```
df = pd.read_csv("titanic.csv")

# First look
df.head(5)          # first 5 rows
df.tail(3)          # last 3 rows
df.shape            # (891, 12) → rows, columns
df.columns          # column names
df.info()           # types + non-null counts
df.describe()       # statistical summary

# Selecting data
df["Age"]            # single column (Series)
df[["Name", "Age"]]  # multiple columns (DataFrame)
df.iloc[0]          # row by position
df.loc[0, "Age"]     # value by label

# Filtering rows
adults = df[df["Age"] >= 18]
survivors = df[df["Survived"] == 1]
female_survivors = df[(df["Sex"] == "female") &
                      (df["Survived"] == 1)]
```

Quick Reference

`read_csv()`

Load data from CSV file

`head()` / `tail()`

Preview first/last rows

`info()`

Column types & null counts

`describe()`

Stats: mean, std, min, max

`iloc` / `loc`

Row/column selection

`Boolean mask`

Filter rows by condition

Data Cleaning & Preprocessing

80% of data science is cleaning — get comfortable with it!

```
python data_cleaning.py
```

```
# Handling missing values
df.isnull().sum()          # count NaN per column
df.dropna()                # drop rows with any NaN
df.fillna(0)               # fill NaN with 0
df["Age"].fillna(df["Age"].mean(), inplace=True)

# Remove duplicates
df.duplicated().sum()      # count duplicate rows
df.drop_duplicates(inplace=True)

# Rename columns
df.rename(columns={"PassengerId": "ID"}, inplace=True)

# Change data types
df["Age"] = df["Age"].astype(int)
df["Survived"] = df["Survived"].astype(bool)

# Reset index after filtering
df.reset_index(drop=True, inplace=True)

# String operations
df["Name"] = df["Name"].str.strip().str.lower()
```

Common Data Problems

? Missing Values

Drop rows/cols or impute with mean/median/mode

🔄 Duplicate Rows

Use drop_duplicates() after investigation

⚠️ Wrong Data Type

Convert with astype() — e.g. str→int

✎ Inconsistent Text

Normalize with str.lower(), str.strip()

Data Visualization — Matplotlib & Seaborn

Turning numbers into insights through visual storytelling

```
python visualization.py
```

```
import matplotlib.pyplot as plt
import seaborn as sns

# Line chart
plt.figure(figsize=(8,4))
plt.plot([1,2,3,4], [10,25,18,30], marker='o')
plt.title("Monthly Sales"); plt.xlabel("Month")
plt.ylabel("Revenue"); plt.grid(True)
plt.show()

# Bar chart
plt.bar(["A","B","C","D"], [85,92,78,95], color='steelblue')
plt.title("Student Scores"); plt.show()

# Scatter plot
plt.scatter(df["Age"], df["Score"], alpha=0.6)
plt.title("Age vs Score"); plt.show()

# Seaborn — statistical plots
sns.histplot(df["Age"], kde=True) # distribution
sns.boxplot(x="Gender", y="Score", data=df) # box plot
sns.heatmap(df.corr(), annot=True, cmap="coolwarm") # correlation
```

Chart Types & When to Use

Line Chart

Trends over time

Bar Chart

Compare categories

Scatter Plot

Relationship between 2 variables

Histogram

Distribution of a variable

Box Plot

Spread & outliers by group

Heatmap

Correlation matrix

Exploratory Data Analysis (EDA)

Asking smart questions before building any model

```
python eda_demo.py
```

```
import pandas as pd
import seaborn as sns, matplotlib.pyplot as plt
```

```
df = pd.read_csv("titanic.csv")
```

```
# — STEP 1: Shape & Types —————
```

```
print(df.shape, df.dtypes)
```

```
# — STEP 2: Summary Stats —————
```

```
print(df.describe())
```

```
# — STEP 3: Missing Values —————
```

```
print(df.isnull().sum() / len(df) * 100)  # % missing
```

```
# — STEP 4: Distribution —————
```

```
sns.histplot(df["Age"].dropna(), kde=True)
```

```
plt.show()
```

```
# — STEP 5: Correlations —————
```

```
sns.heatmap(df.corr(numeric_only=True), annot=True)
```

```
plt.show()
```

```
# — STEP 6: Relationships —————
```

EDA Checklist

1

Understand the Data

Shape, columns, data types

2

Summary Statistics

Mean, median, std, min, max

3

Missing Values

Identify and handle NaN data

4

Distributions

Histograms, box plots per feature

5

Correlations

Which features are related?

6

Feature Relationships

How does X affect the target Y?

Day 2 - Key Takeaways

You now have the Python foundation to start building AI programs!

- 1 Python's clean syntax and ecosystem make it ideal for AI development
- 2 Variables store data, Python is dynamically typed (no explicit declaration needed)
- 3 Control flow (if/for/while) lets programs make decisions and repeat operations
- 4 Functions promote code reuse — define once, call many times
- 5 NumPy enables fast, efficient array/matrix operations critical for ML computations

Day 2 - Key Takeaways

You can now load, clean, and visualize real-world datasets!

- 1 Pandas DataFrames are your primary tool for tabular data like Excel but in Python
- 2 `read_csv()`, `head()`, `info()` and `describe()` are your first steps with any new dataset
- 3 Data cleaning (nulls, duplicates, types) is essential before any analysis or modeling
- 4 Matplotlib gives low-level control; Seaborn adds statistical plots with less code
- 5 EDA is a structured process: understand → summarize → clean → visualize → hypothesize